UNIT 2

Student Record and Academic Management System

Data Organization and Academic Record Management using Python Collections

Project Overview

This project uses Python strings, lists, tuples, sets, and dictionaries to store and manage student records. The system can add, search, update, delete, display, and report on student academic information.

Seven-Stage Engineering Workflow

1. Problem Understanding — identify inputs, outputs, data fields, and suitable data structures.
2. Algorithm Design — write plain-language numbered steps.
3. Flowchart — draw the logic using Start/Stop, process, and decision symbols.
4. Pseudocode — describe the solution in structured English.
5. Python Implementation — translate the pseudocode into Python.
6. Testing — test normal, boundary, invalid, duplicate, and empty inputs.
7. Documentation — add comments, test cases, observations, screenshots, and README.

Data Structure Selection

• Strings: student names, departments, addresses, email IDs, and roll-number text.

• Lists: student records, subject lists, marks, and attendance values.

• Tuples: fixed date-of-birth components or registration details.

• Sets: unique departments, subjects, and clubs.

• Dictionaries: one complete student profile with labelled fields.

Problem Analysis

• Add student: accept student details, reject duplicate roll numbers, then save a dictionary in the records list.

• Search student: compare the requested roll number against each record.

• Update record: find the student and update the selected field.

• Delete record: find and remove the matching student dictionary.

• Calculate average: sum the marks and divide by the number of marks.

• Find highest scorer: compare students' average marks.

• List by department: filter records by department name.

• Count students: count records in the list.

Algorithms, Flowchart Logic, and Pseudocode

Add Student — Algorithm

• Accept student details.

• Check whether roll number already exists.

• If duplicate, show a warning; otherwise create a dictionary and append it to the list.

• Display a confirmation message.

```
START → Read student details → Roll number exists?
YES: Display 'Student already exists' → STOP
NO: Create profile dictionary → Append to records → Display success → STOP
```

```
START
  READ student details
  IF roll number exists THEN
      DISPLAY 'Student already exists'
  ELSE
      CREATE student profile dictionary
      APPEND profile TO student_records
      DISPLAY 'Student added successfully'
  END IF
STOP
```

Search Student — Algorithm and Pseudocode

```
START → Read roll number → Search each record
If match: display profile and stop searching
If no match: display 'Student Not Found' → STOP
```

```
START
  READ target_roll_number
  SET found = False
  FOR EACH student IN student_records
      IF student roll number matches target THEN
          DISPLAY student details
          SET found = True
          EXIT LOOP
      END IF
  END FOR
  IF found == False THEN DISPLAY 'Student Not Found'
STOP
```

Update Student — Pseudocode

```
START
  READ roll number
  FIND matching student
  IF found THEN
      READ field and new value
      UPDATE selected field
      DISPLAY 'Record updated successfully'
  ELSE
      DISPLAY 'Student Not Found'
  END IF
STOP
```

Delete Student — Pseudocode

```
START
  READ roll number
  FIND matching student
  IF found THEN
      REMOVE student from student_records
      DISPLAY 'Record deleted successfully'
  ELSE
      DISPLAY 'Student Not Found'
  END IF
STOP
```

Calculate Average — Pseudocode

```
START
  READ marks
  IF marks list is empty THEN
      DISPLAY 'No marks available'
  ELSE
      SET average = sum(marks) / number of marks
      DISPLAY average
  END IF
STOP
```

Working with Strings, Lists, Tuples, Sets, and Dictionaries

```
name = "  gauri walunj  "
print(name.strip().title())  # Gauri Walunj

marks = [78, 85, 91]
marks.append(88)
marks.sort()
print(marks)
print(sum(marks) / len(marks))

dob = (12, 5, 2008)  # tuple: fixed grouped details
day, month, year = dob

subjects = ["Python", "Maths", "Python"]
print(set(subjects))  # unique subjects

student = {"roll_number": "AIDS101", "name": "Gauri Walunj",
           "department": "AIDS", "marks": [82, 76, 91],
           "attendance": 90, "subjects": ["Python", "Maths"]}
student["attendance"] = 92
print(student.get("name"))
```

APPENDIX A — COMPLETE PYTHON SOURCE CODE

Save this code as student_management.py and run it with Python 3. It is a menu-driven system demonstrating all five collection types.

```
# Student Record and Academic Management System
student_records = []

def find_student(roll_number):
    target = roll_number.strip().upper()
    for student in student_records:
        if student["roll_number"] == target:
            return student
    return None

def read_marks():
    while True:
        try:
            marks = [float(x.strip()) for x in input("Marks (comma-separated): ").split(",") if x.strip()]
            if marks and all(0 <= x <= 100 for x in marks):
                return marks
            print("Enter one or more marks from 0 to 100.")
        except ValueError:
            print("Enter numbers separated by commas.")

def add_student():
    roll = input("Roll number: ").strip().upper()
    if not roll:
        print("Roll number cannot be empty."); return
    if find_student(roll):
        print("Student already exists."); return
    name = input("Name: ").strip().title()
    dept = input("Department: ").strip().upper()
    if not name or not dept:
        print("Name and department are required."); return
    subjects = [x.strip().title() for x in input("Subjects (comma-separated): ").split(",") if x.strip()]
    marks = read_marks()
    while True:
        try:
            attendance = float(input("Attendance percentage (0-100): "))
            if 0 <= attendance <= 100: break
            print("Attendance must be from 0 to 100.")
        except ValueError:
            print("Enter a valid number.")
    contact = input("Contact/email: ").strip()
    dob = tuple(input("Date of birth (DD-MM-YYYY): ").strip().split("-"))
    student_records.append({
        "roll_number": roll, "name": name, "department": dept,
        "subjects": subjects, "marks": marks, "attendance": attendance,
        "contact": contact, "date_of_birth": dob
```

APPENDIX A — COMPLETE PYTHON SOURCE CODE (continued)

```
    })
    print("Student added successfully.")

def average(student):
    return sum(student["marks"]) / len(student["marks"]) if student["marks"] else None

def display_student(s):
    print("-" * 40)
    for key, value in s.items():
        print(f"{key}: {value}")
    avg = average(s)
    print("average_marks:", f"{avg:.2f}" if avg is not None else "No marks")
    print("-" * 40)

def search_student():
    s = find_student(input("Roll number to search: "))
    if s: display_student(s)
    else: print("Student Not Found.")

def update_student():
    s = find_student(input("Roll number to update: "))
    if not s:
        print("Student Not Found."); return
    print("1.Name  2.Department  3.Attendance  4.Contact")
    choice = input("Field: ").strip()
    if choice == "1":
        value = input("New name: ").strip()
        if value: s["name"] = value.title()
        else: print("Name cannot be empty."); return
    elif choice == "2":
        value = input("New department: ").strip()
        if value: s["department"] = value.upper()
        else: print("Department cannot be empty."); return
    elif choice == "3":
        try:
            value = float(input("New attendance (0-100): "))
            if not 0 <= value <= 100: print("Invalid attendance."); return
            s["attendance"] = value
        except ValueError:
            print("Invalid number."); return
    elif choice == "4":
        s["contact"] = input("New contact/email: ").strip()
    else:
        print("Invalid choice."); return
    print("Record updated successfully.")
```

APPENDIX A — COMPLETE PYTHON SOURCE CODE (continued)

```
def delete_student():
    s = find_student(input("Roll number to delete: "))
    if s:
        student_records.remove(s)
        print("Record deleted successfully.")
    else: print("Student Not Found.")

def display_all():
    if not student_records:
        print("No student records available."); return
    for s in student_records: display_student(s)

def calculate_average():
    s = find_student(input("Roll number: "))
    if not s: print("Student Not Found."); return
    avg = average(s)
    print("No marks available." if avg is None else f"Average: {avg:.2f}")

def highest_scorer():
    eligible = [s for s in student_records if s["marks"]]
    if not eligible:
        print("No records with marks available."); return
    top = max(eligible, key=average)
    print(f"Highest scorer: {top['name']} ({top['roll_number']}) — {average(top):.2f}")

def list_by_department():
    dept = input("Department: ").strip().upper()
    matches = [s for s in student_records if s["department"] == dept]
    if matches:
        for s in matches: display_student(s)
    else: print("No students found in this department.")

def unique_departments():
    # Set removes duplicates automatically.
    depts = {s["department"] for s in student_records}
    print("Unique departments:", sorted(depts) if depts else "None")

def main():
    while True:
        print("\\n===== STUDENT RECORD MANAGEMENT SYSTEM =====")
        print("1 Add  2 Search  3 Update  4 Delete  5 Display all")
        print("6 Average  7 Highest scorer  8 By department")
        print("9 Unique departments  10 Count  0 Exit")
        choice = input("Choice: ").strip()
```

APPENDIX A — COMPLETE PYTHON SOURCE CODE (continued)

```
        if choice == "1": add_student()
        elif choice == "2": search_student()
        elif choice == "3": update_student()
        elif choice == "4": delete_student()
        elif choice == "5": display_all()
        elif choice == "6": calculate_average()
        elif choice == "7": highest_scorer()
        elif choice == "8": list_by_department()
        elif choice == "9": unique_departments()
        elif choice == "10": print("Total students:", len(student_records))
        elif choice == "0":
            print("Thank you for using the system."); break
        else: print("Invalid choice. Try again.")

if __name__ == "__main__":
    main()
```

Testing and Expected Results

• Add a new student with marks 70, 80, 90: record should be added and average should be 80.00.

• Try adding the same roll number again: the program should show 'Student already exists'.

• Search an unknown roll number: the program should show 'Student Not Found'.

• Enter text instead of marks or marks outside 0–100: the program should request valid input.

• Display all records before adding a student: the program should show 'No student records available'.

• Filter by a department with no matching records: the program should show a no-match message.

Submission Checklist

• Problem Analysis Report

• Data Structure Selection Report

• Algorithm Design Document

• Flowchart Collection

• Pseudocode Document

• String, List, Tuple, Set, and Dictionary Exercises

• Integrated Python Application

• Testing Report and Trace Table

• Screenshots and README

• Updated GitHub Repository